

Conferencia 3: demostrar que un algoritmo es correcto

Diseño y Análisis de Algoritmos · Ciencia de la Computación · MatCom · 2026

La conferencia anterior terminó con una advertencia. Una batería de pruebas pasó en verde sobre un algoritmo incorrecto, y lo único que decía dónde buscar el error era la demostración. Esta conferencia trata de cómo se escribe esa demostración. Vamos a ver tres herramientas, cada una sobre un algoritmo cuya correctitud no se ve a simple vista, y al final vas a saber qué tipo de error atrapa cada una.

1. Qué significa "correcto"

Un algoritmo es correcto respecto a una **especificación**, y la especificación es un contrato con dos partes. La **precondición** dice qué entradas acepta el algoritmo. La **postcondición** dice qué relación tiene que haber entre la entrada y la salida. La búsqueda binaria, por ejemplo, tiene como precondición que el arreglo esté ordenado, y como postcondición que devuelve una posición donde está x , o -1 si x no está. Sobre un arreglo desordenado la búsqueda binaria no es incorrecta: simplemente el contrato no dice nada.

Demostrar que un algoritmo cumple su contrato son dos demostraciones separadas:

- **Correctitud parcial:** si el algoritmo termina, la salida cumple la postcondición.
- **Terminación:** el algoritmo termina para toda entrada que cumpla la precondición.

La **correctitud total** es la suma de las dos. Hay que hacerlas por separado porque ninguna implica la otra. Un programa que es solo un ciclo infinito es parcialmente correcto para cualquier especificación, porque nunca devuelve nada, y por tanto nunca devuelve algo incorrecto.

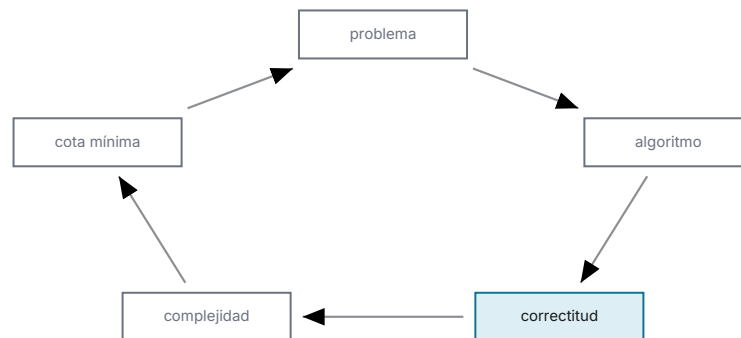


Figura 1. El ciclo de la conferencia 2. Hoy nos quedamos en la tercera etapa.

Para esas dos demostraciones el curso usa tres herramientas, y cada una responde una pregunta distinta:

1. **Invariante:** ¿qué se mantiene verdadero en cada vuelta del ciclo? Se demuestra en tres pasos. Inicialización: es verdadero antes de la primera vuelta. Mantenimiento: si es verdadero antes de una vuelta, lo sigue siendo después. Conclusión: al terminar, el invariante junto con la condición de salida implica la postcondición. Da la correctitud parcial.
2. **Potencial:** ¿qué cantidad entera no negativa baja en cada vuelta? Como no puede bajar para siempre, el ciclo termina, y el valor inicial acota el número de vueltas. Da la terminación.
3. **Contradicción sobre el primer fallo:** si el resultado fuera malo, habría un primer momento de la ejecución en que algo salió mal. Se muestra que ese momento no puede existir. Da propiedades globales del resultado, como la estabilidad y la optimalidad de la segunda mitad de la clase.

Las tres son la inducción de la conferencia 2 con otra ropa: el invariante es inducción sobre el número de vueltas, el potencial es inducción sobre los naturales en sentido descendente, y el primer fallo es inducción fuerte escrita al revés.

Ya las usaste sin nombrarlas. La búsqueda binaria que viste en Estructuras de Datos funciona por el invariante "si x está en el arreglo, está entre lo y hi ", y termina porque el potencial $hi - lo$ baja en cada vuelta. El ejercicio 7 cambia un $+ 1$ y el potencial deja de bajar.

2. El problema de la mayoría

Mayoría. Dado un arreglo A de n elementos, devolver el elemento que aparece más de $n/2$ veces, o decir que no hay ninguno.

Una precisión cambia todo el problema: **los elementos solo se pueden comparar por igualdad**. No hay un orden entre ellos, así que no se pueden ordenar. Piensa en votos por candidatos, en identificadores opacos o en registros enormes, donde hashear cada uno tampoco es gratis. Queremos tiempo $O(n)$ y memoria extra $O(1)$: un número constante de variables, sin importar n .

Las soluciones que salen primero no llegan:

- Contar las apariciones de cada elemento cuesta $\Theta(n^2)$ comparaciones.
- Ordenar y mirar el elemento del medio cuesta $O(n \log n)$, y además necesita un orden que no tenemos.
- Una tabla hash de cuentas cuesta $O(n)$ tiempo esperado, pero $O(n)$ memoria.

Cómo se le ocurre a uno. La observación es de cancelación. Si tachas dos elementos distintos del arreglo, la mayoría sigue siendo mayoría en lo que queda. Si la mayoría m aparece $k > n/2$ veces, entre los dos elementos tachados hay a lo sumo una copia de m , así que quedan al menos $k - 1$ copias entre $n - 2$ elementos, y $k - 1 > (n - 2)/2$. Puedo seguir tachando parejas de distintos hasta que no queden, y lo que sobreviva son copias de un solo elemento. Si había mayoría, tiene que ser ese.

Hazlo a mano con la fila B B A C A A B A A. Tacha A con uno de los B, después C con el otro B, después la última B con una A. Sobreviven tres A, y en efecto A aparece 5 veces de 9. El algoritmo que sigue hace exactamente eso sin guardar la fila tachada: solo recuerda qué elemento sobrevive y cuántas copias quedan.

3. Boyer-Moore: el algoritmo

El algoritmo guarda un candidato y un contador. Recorre el arreglo una vez. Si el contador está en 0, el elemento actual pasa a ser candidato. Si el elemento coincide con el candidato, el contador sube. Si no coincide, el contador baja: el elemento actual se cancela con una de las copias del candidato. Después hay una segunda pasada que cuenta el candidato y comprueba que de verdad es mayoría.

```
def mayoria(A):
    candidato, c = None, 0
    for x in A:
        if c == 0:
            candidato, c = x, 1
        elif x == candidato:
            c += 1
        else:
            c -= 1
    # segunda pasada: verificar
    return candidato if 2 * A.count(candidato) > len(A) else None

print(mayoria(list("BBACAABAA")), mayoria(list("ABC")))
```

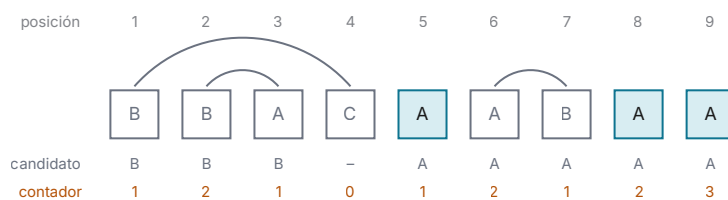


Figura 2. La primera pasada sobre B B A C A A B A A. Cada arco une un elemento con la copia del candidato con la que se canceló. Sobreviven tres copias de A, tantas como marca el contador al final.

La [figura 2](#) muestra el estado en cada paso. Fíjate en la posición 4: el contador vuelve a 0 y el candidato B desaparece, aunque en ese momento B es el elemento más frecuente del prefijo. El candidato no es "el que más aparece hasta ahora". Es otra cosa, y la siguiente sección dice qué.

El algoritmo es de Robert Boyer y J Strother Moore. Lo escribieron en 1980 y lo publicaron en 1991, y lo demostraron correcto con su propio demostrador automático de teoremas, lo que no deja de ser apropiado para una clase sobre demostraciones.

4. Boyer-Moore: el invariante

Para demostrar un algoritmo con un ciclo, hay que encontrar el invariante. Esta es la parte creativa de la demostración, y casi nunca sale a la primera.

El invariante que uno escribe primero es la frase que queremos al final, aplicada a cada prefijo: si el prefijo procesado tiene mayoría, el candidato es esa mayoría. Esta frase es verdadera. Pero no se puede demostrar por inducción tal como está. Intenta el paso de mantenimiento: supón que vale para el prefijo de largo i y procesa el elemento $i + 1$. Si el prefijo de largo i no tiene mayoría, la hipótesis no dice absolutamente nada sobre el candidato, y sin embargo el prefijo de largo $i + 1$ puede tener mayoría. En la fila de la [figura 2](#) pasa en el último paso: el prefijo de largo 8 tiene cuatro A de ocho, que no es mayoría, y el de largo 9 sí la tiene.

Un invariante tiene que ser **inductivo**, no solo verdadero: tiene que dar información suficiente sobre el estado para poder demostrarse a sí mismo en el paso siguiente. Cuando uno no lo es, hay que **reforzarlo**, es decir, afirmar algo más fuerte que sí se sostenga solo. Parece paradójico que sea más fácil demostrar algo más fuerte, pero es lo normal en inducción: la hipótesis más fuerte es también una herramienta más fuerte.

El invariante reforzado es la idea de cancelación de la sección 2, escrita como afirmación sobre el estado. Después de procesar el prefijo $A[1..i]$:

el prefijo se puede partir en parejas de elementos distintos más c copias del candidato, donde c es el valor del contador.

Inicialización. Antes de la primera vuelta el prefijo es vacío y $c = 0$: cero parejas y cero copias.

Mantenimiento. Supongamos el invariante para $A[1..i]$ y procesemos $x = A[i + 1]$. Hay tres casos, uno por rama del `if`:

- Si $c = 0$, el prefijo es solo parejas. El algoritmo hace candidato a x con $c = 1$, y la partición nueva es la misma más una copia de x .
- Si x es el candidato, c sube en 1 y x es una copia más.

- Si x es distinto del candidato y $c > 0$, emparejamos x con una de las c copias. Es una pareja de elementos distintos, y quedan $c - 1$ copias, que es lo que el algoritmo deja en el contador.

Conclusión. Al terminar, A entero se parte en $(n - c)/2$ parejas de distintos y c copias del candidato. Supongamos que m es mayoría, o sea que aparece más de $n/2$ veces, y que el candidato final no es m . Entonces m solo aparece en las parejas, y a lo sumo una vez por pareja, porque los dos elementos de una pareja son distintos. En total, a lo sumo $(n - c)/2 \leq n/2$ veces. Contradicción. $\square$

El teorema que acabamos de demostrar es: **si hay mayoría, es el candidato**. Guarda la forma exacta de esa frase, con su "si", porque la sección 5 depende de ella. La segunda pasada completa el algoritmo: si el candidato no es mayoría, el teorema dice que nadie lo es, y la respuesta correcta es que no hay mayoría.

Un invariante también se puede ejecutar. Para comprobar la partición en cada paso hace falta saber cuándo un multiconjunto se parte en parejas de distintos, y la respuesta es un lema corto: cuando tiene tamaño par y ningún elemento ocupa más de la mitad. (Ordénalo y empareja la posición i con la $i + k$, donde $2k$ es el tamaño: si alguna pareja fuera de iguales, ese elemento ocuparía más de k posiciones seguidas.)

```
def se_parte_en_parejas(S):
    return len(S) % 2 == 0 and all(2 * k <= len(S) for k in Counter(S).values())

def primera_pasada_vigilada(A):
    candidato, c = None, 0
    for i, x in enumerate(A):
        if c == 0:
            candidato, c = x, 1
        elif x == candidato:
            c += 1
        else:
            c -= 1
    resto = Counter(A[:i + 1])
    resto[candidato] -= c # quitar las c copias del candidato
    assert resto[candidato] >= 0 and se_parte_en_parejas(list(resto.elements()))
    return candidato, c

random.seed(3)
prefijos = 0
for _ in range(3000):
    A = [random.choice("ABC") for _ in range(random.randint(1, 12))]
    primera_pasada_vigilada(A)
    prefijos += len(A)
print("invariante comprobado en", prefijos, "prefijos, sin fallos")
```

invariante comprobado en 19342 prefijos, sin fallos

Esto no reemplaza a la demostración: comprueba miles de prefijos, no todos. Pero es el experimento que conviene correr **antes** de intentar la demostración. Si el invariante que escribiste es falso, el assert te lo dice en un segundo, y no pierdes una tarde intentando demostrar algo que no es verdad.

Complejidad. Dos pasadas, $O(n)$ tiempo, y memoria $O(1)$: un candidato y un contador. **Cota mínima.** Cualquier algoritmo tiene que leer $\Omega(n)$ posiciones. Toma n impar y un arreglo con $(n - 1)/2$ copias de a y el resto de elementos distintos entre sí. No tiene mayoría. Si un algoritmo no lee alguna de las posiciones que no tienen a , cambiar esa posición por a produce un arreglo con mayoría, sobre el que el algoritmo hace exactamente lo mismo y responde lo mismo. Así que tiene que leer las $(n + 1)/2$ posiciones sin a , y Boyer-Moore es óptimo.

5. El error que la demostración anuncia

La segunda pasada parece un desperdicio. Mira el contador: al final, c cuenta las copias del candidato que sobrevivieron. Es tentador pensar que si sobrevivió alguna, el candidato es mayoría, y ahorrarse la segunda pasada.

```
def mayoria_atajo(A):
    candidato, c = None, 0
    for x in A:
        if c == 0:
            candidato, c = x, 1
        elif x == candidato:
            c += 1
        else:
            c -= 1
    return candidato if c > 0 else None
```

La mitad del atajo es correcta. Si hay mayoría, el contador final no puede ser 0: con $c = 0$ la conclusión de la sección 4 da que cualquier m aparece a lo sumo $n/2$ veces. Así que $c = 0$ implica que no hay mayoría. Lo falso es la recíproca: $c > 0$ no implica que haya mayoría. El contraejemplo mínimo es A B C: A y B se cancelan, C sobrevive con contador 1, y ninguno de los tres es mayoría.

Lo interesante es cómo se ve este error en las pruebas. Comparemos las dos versiones contra un oráculo que cuenta todo, con cuatro generadores de arreglos.

```
def oraculo(A):
    for x, k in Counter(A).items():
        if 2 * k > len(A):
            return x
    return None

def plantada(n):
    # el generador que casi todo el mundo escribe
    m = random.choice("ABCDE")
    A = [m] * (n // 2 + 1) + [random.choice("ABCDE") for _ in range(n - n // 2 - 1)]
    random.shuffle(A)
    return A

generadores = {
    "mayoría plantada": plantada,
    "binario": lambda n: [random.choice("AB") for _ in range(n)],
    "3 símbolos": lambda n: [random.choice("ABC") for _ in range(n)],
    "5 símbolos": lambda n: [random.choice("ABCDE") for _ in range(n)],
}

random.seed(5)
fallos = {}
print(f"{'generador':>18} {'atajo':>8} {'dos pasadas':>12}")
for nombre, generar in generadores.items():
    casos = [generar(random.randint(1, 15)) for _ in range(4000)]
    f_atajo = sum(mayoria_atajo(A) != oraculo(A) for A in casos) / len(casos)
    f_bm = sum(mayoria(A) != oraculo(A) for A in casos) / len(casos)
    fallos[nombre] = (100 * f_atajo, 100 * f_bm)
    print(f"{'nombre':>18} {100 * f_atajo:>7.1f}% {100 * f_bm:>11.1f}%")
```

generador	atajo	dos pasadas
mayoría plantada	0.0%	0.0%
binario	0.0%	0.0%
3 símbolos	34.0%	0.0%
5 símbolos	47.1%	0.0%

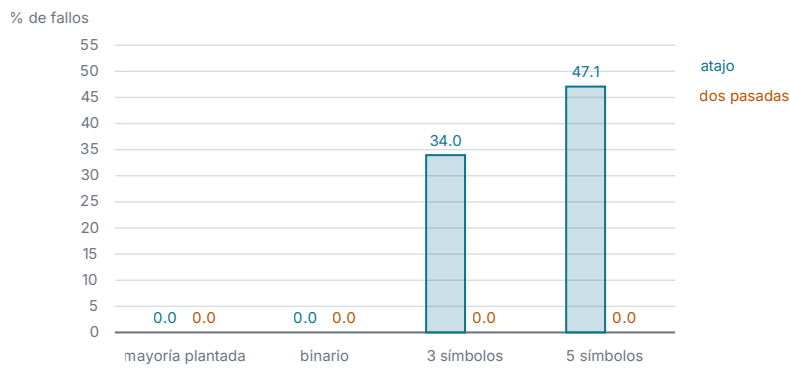


Figura 3. Porcentaje de arreglos en que cada versión se equivoca, según cómo se generaron las pruebas. Con los dos primeros generadores el atajo es indistinguible del algoritmo correcto.

Los dos primeros generadores no encuentran nada. El de mayoría plantada no puede encontrarlo: todos sus arreglos tienen mayoría, y sobre esos el atajo es correcto por la mitad buena del argumento. El binario es más sutil. Con dos símbolos el contador final es exactamente la diferencia entre las dos cuentas (ejercicio 3), así que $c > 0$ significa que un símbolo aparece más que el otro, que con dos símbolos es lo mismo que ser mayoría. El atajo es **correcto** sobre alfabetos binarios, y una batería de pruebas binaria no puede atraparlo nunca. Con tres símbolos o más falla en una fracción grande de los casos.

Compara con la conferencia 2. Allí el error del lema de la franja casi no se veía, porque las entradas que lo disparan son raras: cerca del 1 % de las pequeñas y ninguna de las grandes. Aquí las pruebas sí lo atrapan, pero solo si alguien prueba el caso en que no hay mayoría. ¿Quién te avisa de que ese caso existe y es distinto? La demostración: su conclusión empieza con "si hay mayoría", y lo que no cubre esa hipótesis es exactamente lo que el atajo hace mal. **Leer la hipótesis de un teorema es leer la lista de casos que hay que probar.**

6. El emparejamiento estable

Cambiamos de problema, y de herramienta principal.

Emparejamiento estable. Hay n **proponentes** $a_1, \dots, a_n$ y n **receptores** $b_1, \dots, b_n$. Cada uno tiene una lista de preferencias estricta y completa sobre todos los del otro lado. Un emparejamiento perfecto M empareja a cada proponente con un receptor distinto. M es **inestable** si hay un proponente a y un receptor b , no emparejados entre sí, tales que a prefiere b a su pareja en M , y b prefiere a a su pareja en M . A ese par se le llama **pareja bloqueante**: los dos ganarían dejando a sus parejas y yéndose juntos. Buscamos un emparejamiento sin parejas bloqueantes.

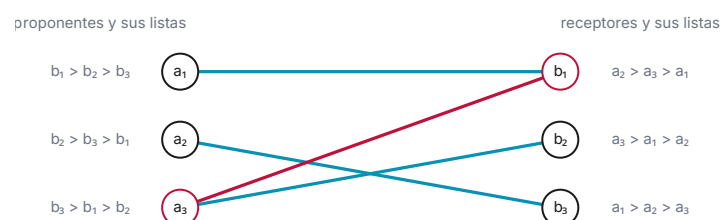


Figura 4. Un emparejamiento inestable, en azul. Tiene una sola pareja bloqueante, en rojo: a_3 prefiere b_1 a su pareja, y b_1 prefiere a_3 a la suya.

La figura 4 muestra una instancia de 3×3 con un emparejamiento inestable. Que exista un emparejamiento estable para toda instancia no es obvio, y de hecho depende de que haya dos lados. En la versión con un solo grupo, el problema de los **compañeros de cuarto** (cada persona ordena a todas las demás y

hay que emparejarlas de a dos), hay instancias de cuatro personas sin ningún emparejamiento estable. El ejercicio 4 pide construir una. Así que la existencia es algo que hay que demostrar, y en este caso la demostración va a ser el propio algoritmo.

El problema no es un juguete. La asignación de médicos recién graduados a plazas de residencia en los hospitales de Estados Unidos funciona con este esquema desde los años 50. David Gale y Lloyd Shapley lo formalizaron en 1962, y en 2012 Shapley y Alvin Roth recibieron el Nobel de Economía por la teoría y el diseño de estos mercados.

Cómo se le ocurre a uno. El primer algoritmo que sale es reparar: empezar con cualquier emparejamiento y, mientras haya una pareja bloqueante (a, b) , emparejar a con b y dejar a sus dos antiguas parejas juntas. Cada reparación elimina una pareja bloqueante, pero puede crear otras, y Donald Knuth mostró en 1976 que esta reparación local puede **ciclar para siempre**, volviendo a un emparejamiento que ya se había visitado (el ejercicio 6 pide una instancia). La reparación no tiene un potencial: nada avanza siempre en la misma dirección. Eso es lo que hay que buscar, un procedimiento donde algo se mueva en un solo sentido.

7. Gale-Shapley: el algoritmo y su terminación

La idea de Gale y Shapley es la **aceptación diferida**. Los proponentes proponen en orden, de su favorito hacia abajo. Un receptor nunca dice que sí de forma definitiva: se queda con la mejor propuesta que ha recibido hasta ahora y la cambia si llega una mejor.

Mientras haya un proponente libre, ese proponente le propone al mejor receptor de su lista al que todavía no le ha propuesto. Si el receptor está libre, acepta. Si ya tiene pareja y prefiere al nuevo, lo acepta y su pareja anterior queda libre. Si no, lo rechaza.

El algoritmo es **no determinista**: no dice cuál proponente libre actúa primero. Todo lo que demos demos tiene que valer para cualquier elección, y la implementación deja elegir la estrategia (cola, pila o al azar) para poder comprobarlo después.

```
def gale_shapley(pref_a, pref_b, orden="cola", semilla=0, traza=None):
    n = len(pref_a)
    rango = [[0] * n for _ in range(n)]      # rango[b][a]: lugar de a en la lista de b
    for b in range(n):
        for lugar, a in enumerate(pref_b[b]):
            rango[b][a] = lugar
    siguiente = [0] * n                       # a qué lugar de su lista va a proponer a
    pareja_de_b = [None] * n
    libres = deque(range(n))
    azar = random.Random(semilla)
    propuestas = 0

    while libres:
        if orden == "azar":                   # rotar cuesta O(n); solo es para las pruebas
            libres.rotate(azar.randrange(len(libres)))
        a = libres.popleft() if orden == "cola" else libres.pop()
        b = pref_a[a][siguiente[a]]
        siguiente[a] += 1
        propuestas += 1
        actual = pareja_de_b[b]
        if actual is None:
            pareja_de_b[b] = a
        elif rango[b][a] < rango[b][actual]:
            pareja_de_b[b] = a
            libres.append(actual)
        else:
            libres.append(a)
        if traza is not None:
            traza.append((a, b, actual, pareja_de_b[b]))
```

```

M = [None] * n
for b, a in enumerate(pareja_de_b):
    M[a] = b
return M, propuestas

```

PROPONENTE	SU LISTA	RECEPTOR	SU LISTA
a_1	$b_4 > b_1 > b_3 > b_2$	b_1	$a_2 > a_3 > a_4 > a_1$
a_2	$b_4 > b_1 > b_2 > b_3$	b_2	$a_1 > a_3 > a_4 > a_2$
a_3	$b_3 > b_4 > b_2 > b_1$	b_3	$a_4 > a_2 > a_1 > a_3$
a_4	$b_1 > b_3 > b_4 > b_2$	b_4	$a_4 > a_1 > a_3 > a_2$

PASO	PROPONENTE	RECEPTOR	TENÍA	SE QUEDA CON	QUÉ PASA
1	a_1	b_4	—	a_1	acepta
2	a_2	b_4	a_1	a_1	rechaza a a_2
3	a_3	b_3	—	a_3	acepta
4	a_4	b_1	—	a_4	acepta
5	a_2	b_1	a_4	a_2	cambia; a_4 queda libre
6	a_4	b_3	a_3	a_4	cambia; a_3 queda libre
7	a_3	b_4	a_1	a_1	rechaza a a_3
8	a_3	b_2	—	a_3	acepta

Resultado: a_1-b_4 , a_2-b_1 , a_3-b_2 , a_4-b_3 , en 8 propuestas.

La traza de arriba es una instancia de 4×4 con la estrategia de cola. Fíjate en dos cosas que la tabla deja ver y que son la base de todo lo que sigue.

- **Un receptor, una vez comprometido, no vuelve a quedar libre, y su pareja solo mejora.** Solo cambia de pareja si llega alguien que prefiere. Mira la columna "se queda con" para cualquier receptor: siempre sube en su lista.
- **Un proponente propone en orden decreciente de preferencia, así que sus parejas solo empeoran.** Cada vez que queda libre, sigue desde donde estaba en su lista.

Los dos son invariantes de una línea, y su demostración es leer el código.

Terminación, por potencial. Sea Φ el número de pares (a, b) tales que a todavía no le ha propuesto a b . Al principio $\Phi = n^2$. Cada vuelta del ciclo es exactamente una propuesta, que nunca se repite porque $\text{siguiente}[a]$ solo avanza, así que Φ baja en 1. Y $\Phi \geq 0$ siempre. El ciclo termina después de a lo sumo n^2 vueltas.

Falta ver que $\text{siguiente}[a]$ nunca se sale de la lista, es decir, que ningún proponente se queda libre después de proponerle a todos. **Termina con un emparejamiento perfecto.** Supongamos que en algún

momento un proponente a está libre y ya le propuso a los n receptores. Cada receptor recibió al menos una propuesta y, por el primer invariante, desde entonces está comprometido. Entonces los n receptores están comprometidos con n proponentes distintos. Pero a está libre, así que solo hay $n - 1$ proponentes comprometidos. Contradicción.

La conferencia 4 usa la palabra **potencial** para otra cosa: una cuenta que paga el costo amortizado de una estructura de datos. Es la misma idea, una cantidad que se mueve en una sola dirección, usada para acotar costo en vez de para demostrar que algo termina.

8. Gale-Shapley: estabilidad

Teorema. El emparejamiento que devuelve Gale-Shapley es estable, para cualquier orden de ejecución.

Demostración. Por contradicción. Sea M el resultado y supongamos que (a, b) es una pareja bloqueante: a prefiere b a $M(a)$, y b prefiere a a $M(b)$. Como a propone en orden decreciente y terminó con $M(a)$, le propuso a b antes que a $M(a)$. En algún momento b rechazó a a , al recibir la propuesta o más tarde, a cambio de alguien que prefería. Por el primer invariante, la pareja de b solo mejora a partir de ahí, así que b prefiere $M(b)$ a a . Contradicción con que (a, b) bloquea. $\square$

Corolario. Toda instancia con dos lados y listas completas y estrictas tiene un emparejamiento estable. El algoritmo **es** la demostración de existencia: una demostración constructiva, que además de decir que existe dice cómo encontrarlo.

Comprobémoslo con un verificador de estabilidad por fuerza bruta, que mira los n^2 pares.

```
def pares_bloqueantes(M, pref_a, pref_b):
    #  $O(n^3)$  por los index, pero es el oráculo: lo que importa es que sea obvio
    pareja_de_b = {b: a for a, b in enumerate(M)}
    return [(a, b) for a in range(len(M)) for b in range(len(M))
            if b != M[a]
            and pref_a[a].index(b) < pref_a[a].index(M[a])
            and pref_b[b].index(a) < pref_b[b].index(pareja_de_b[b])]

def instancia(n, rng):
    return ([rng.sample(range(n), n) for _ in range(n)],
            [rng.sample(range(n), n) for _ in range(n)])

rng = random.Random(8)
print(f"{'n':>5} {'instancias':>11} {'bloqueantes':>12} {'propuestas':>11} {'n^2':>8}")
for n, veces in [(5, 400), (20, 100), (60, 20)]:
    bloqueantes, propuestas = 0, 0
    for _ in range(veces):
        pref_a, pref_b = instancia(n, rng)
        M, k = gale_shapley(pref_a, pref_b, orden="azar", semilla=n)
        bloqueantes += len(pares_bloqueantes(M, pref_a, pref_b))
        propuestas += k
    print(f"{'n':>5} {'veces':>11} {'bloqueantes':>12} {'propuestas / veces':>11.1f} {'n * n':>8}")
```

n	instancias	bloqueantes	propuestas	n^2
5	400	0	8.8	25
20	100	0	61.0	400
60	20	0	243.7	3600

Ninguna pareja bloqueante en más de quinientas instancias, que es lo que el teorema predice. La última columna es la cota del potencial, y la penúltima es lo que pasa en instancias aleatorias: muy por debajo. Con listas al azar, casi todos los proponentes se quedan con uno de sus primeros receptores, y el número de propuestas crece aproximadamente como $n \ln n$. La cota n^2 es de peor caso: hay listas construidas

para que el algoritmo haga del orden de n^2 propuestas, y el análisis tiene que cubrirlas. Otra vez, como en la conferencia 2, el caso típico y la garantía son cosas distintas.

9. Gale-Shapley: optimalidad para quien propone

Una instancia puede tener muchos emparejamientos estables. La instancia de la [figura 4](#) tiene tres:

```
pref_a = [[0, 1, 2], [1, 2, 0], [2, 0, 1]]
pref_b = [[1, 2, 0], [2, 0, 1], [0, 1, 2]]

def todos_estables(pref_a, pref_b):
    n = len(pref_a)
    return [list(M) for M in permutations(range(n))
            if not pares_bloqueantes(list(M), pref_a, pref_b)]

for M in todos_estables(pref_a, pref_b):
    print(" ".join(f"a{a + 1}-b{b + 1}" for a, b in enumerate(M)))
print("Gale-Shapley:", gale_shapley(pref_a, pref_b)[0])
```

```
a1-b1 a2-b2 a3-b3
a1-b2 a2-b3 a3-b1
a1-b3 a2-b1 a3-b2
Gale-Shapley: [0, 1, 2]
```

En el primero cada proponente tiene a su favorito. En el tercero, cada receptor tiene al suyo. El segundo es intermedio. Gale-Shapley devuelve el primero. ¿Es casualidad de esta instancia, y depende del orden en que se elijan los proponentes libres?

Definición. Un receptor b es **pareja válida** de un proponente a si existe algún emparejamiento estable en el que están juntos. $\text{mejor}(a)$ es la pareja válida que a prefiere entre todas.

Teorema. Toda ejecución de Gale-Shapley empareja a cada proponente a con $\text{mejor}(a)$.

Fijate en lo que dice. Primero, que el emparejamiento que forman todos los $\text{mejor}(a)$ es un emparejamiento, sin dos proponentes con el mismo receptor, y además estable, lo cual no es nada obvio. Segundo, que el resultado no depende del orden de ejecución, porque $\text{mejor}(a)$ no depende de él.

Demostración. Por el primer fallo. Como a propone en orden decreciente, basta ver que ningún proponente es rechazado nunca por una pareja válida: entonces llega hasta $\text{mejor}(a)$ y ahí se queda. Supongamos que en alguna ejecución algún proponente sí es rechazado por una pareja válida, y tomemos **el primer rechazo de ese tipo** en el tiempo. Digamos que b rechaza a a a cambio de a' , que b prefiere, y que b es pareja válida de a : existe un emparejamiento estable M' con $(a, b) \in M'$. En M' , el proponente a' está con algún $b' \neq b$.

Ahora miremos a a' en el momento del rechazo. a' le propuso a b , así que antes fue rechazado por todos los receptores que prefiere a b . Esos rechazos ocurrieron antes que el nuestro, que es el primero hecho por una pareja válida, así que ninguno de esos receptores es pareja válida de a' . Pero b' sí lo es, porque está con a' en M' . Entonces b' no está entre los que a' prefiere a b : a' **prefiere** b a b' . Y b **prefiere** a' a a , porque rechazó a a por a' . Así que (a', b) es una pareja bloqueante de M' , que era estable. Contradicción. $\square$

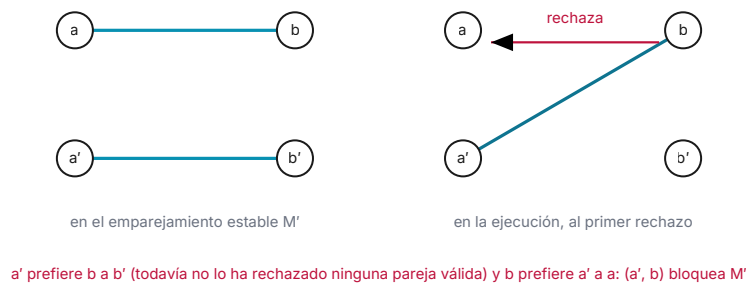


Figura 5. La demostración en un dibujo. A la izquierda, el emparejamiento estable M' donde a está con su pareja válida b . A la derecha, el primer rechazo de la ejecución hecho por una pareja válida. Juntos, (a', b) bloquea M' .

La técnica tiene la forma de la **figura 5**: tomar el **primer** contraejemplo en el tiempo y usar que es el primero, porque todo lo que pasó antes todavía está bien. En la demostración, "es el primero" es lo que dice que a' no fue rechazado por b' . Sin esa elección el argumento no cierra. La misma forma vuelve en el Tema 2, en los argumentos de intercambio de los algoritmos golosos, donde se toma la primera decisión en que el goloso y una solución óptima difieren.

Comprobémoslo en instancias pequeñas, donde se pueden enumerar todos los emparejamientos estables, y con las tres estrategias de ejecución.

```
rng = random.Random(11)
iguales = optimo = 0
for _ in range(300):
    pref_a, pref_b = instancia(5, rng)
    resultados = [gale_shapley(pref_a, pref_b, orden=0, semilla=s)[0]
                  for o, s in [("cola", 0), ("pila", 0), ("azar", 1), ("azar", 2)]]
    iguales += all(M == resultados[0] for M in resultados)
    M = resultados[0]
    estables = todos_estables(pref_a, pref_b)
    optimo += all(pref_a[a].index(M[a]) <= min(pref_a[a].index(E[a]) for E in estables)
                  for a in range(5))
print("mismo resultado con las cuatro ejecuciones:", iguales, "de 300")
print("cada proponente con su mejor pareja válida:", optimo, "de 300")
```

```
mismo resultado con las cuatro ejecuciones: 300 de 300
cada proponente con su mejor pareja válida: 300 de 300
```

El teorema tiene un reverso que el ejercicio 5 pide demostrar: el mismo emparejamiento le da a cada receptor su **peor** pareja válida. Quién propone decide quién gana, y en un mercado real esa es una decisión de diseño con consecuencias para personas de verdad. Gale-Shapley no es neutral, y la demostración de optimalidad es la que lo dice.

10. Complejidad y cota mínima

El potencial acota las vueltas en n^2 . Para que cada vuelta cueste $O(1)$ hacen falta dos estructuras, y las dos están en el código:

- `siguiente[a]`, un puntero por proponente al próximo receptor de su lista.
- `rango[b][a]`, la lista de cada receptor invertida, para que comparar dos pretendientes sea comparar dos números. Sin esa tabla, cada comparación busca en la lista de b y cuesta $O(n)$, y el peor caso sube a $O(n^3)$.

Con las dos, Gale-Shapley cuesta $O(n^2)$ tiempo y $O(n^2)$ memoria. Como en la conferencia 2, la idea es la misma y el costo lo decide la implementación. La entrada son $2n$ listas de largo n , así que el algoritmo es lineal en el tamaño de la entrada.

¿Se puede mejor? No en el peor caso. Cheng Ng y Daniel Hirschberg demostraron en 1990 que encontrar un emparejamiento estable requiere $\Omega(n^2)$ consultas a las listas de preferencias. No lo demostramos aquí, pero con eso Gale-Shapley es óptimo en ese modelo, y el ciclo de la conferencia 2 queda cerrado también para este problema.

11. Qué atrapa cada herramienta

HERRAMIENTA	QUÉ DEMUESTRA	DÓNDE APARECIÓ	QUÉ ERROR ATRAPA
invariante	correctitud parcial	Boyer-Moore	el atajo del contador, porque la conclusión tiene una hipótesis
potencial	terminación	Gale-Shapley	la reparación local, que no tiene nada que baje y cicla
primer fallo	estabilidad, optimalidad	Gale-Shapley	la sospecha de que el resultado depende del orden

Cada herramienta tiene un momento creativo que no se automatiza. En el invariante, es encontrar la afirmación que se sostiene sola, que casi nunca es la primera que uno escribe. En el potencial, es encontrar la cantidad que baja. En el primer fallo, es decidir qué cuenta como fallo, para que "ser el primero" diga algo útil. Por eso vale la pena correr el algoritmo y mirar trazas antes de demostrar, como hicimos con el invariante vigilado de la sección 4. El experimento propone el enunciado, y la demostración lo convierte en teorema.

Las tres vuelven. El potencial de la conferencia 4 paga costo amortizado en vez de contar vueltas. El primer fallo es la forma de los argumentos de intercambio del Tema 2. Y la lección de la sección 5, que la hipótesis de un teorema es una lista de casos, vuelve cada vez que un algoritmo tenga precondiciones que las pruebas no revisan.

12. Resumen

- Correcto quiere decir correcto respecto a una especificación: precondición y postcondición. La correctitud total es correctitud parcial más terminación, y se demuestran por separado.
- Un invariante se demuestra en tres pasos: inicialización, mantenimiento y conclusión. Tiene que ser inductivo, no solo verdadero, y a veces hay que reforzarlo.
- Boyer-Moore encuentra la mayoría en $O(n)$ tiempo y $O(1)$ memoria por cancelación: el prefijo se parte en parejas de distintos más c copias del candidato.
- El teorema es "si hay mayoría, es el candidato". La segunda pasada no sobra: el atajo que la omite pasa todas las pruebas con mayoría plantada y todas las binarias.
- Un potencial es una cantidad entera no negativa que baja en cada vuelta, y da la terminación. En Gale-Shapley es el número de propuestas que faltan.
- Gale-Shapley devuelve un emparejamiento estable, lo que demuestra que siempre existe, y le da a cada proponente su mejor pareja válida en cualquier orden de ejecución.
- La demostración de optimalidad toma el primer rechazo hecho por una pareja válida y usa que es el primero. Es la forma de los argumentos de intercambio del Tema 2.

- Gale-Shapley cuesta $O(n^2)$ con la tabla de rangos, que es óptimo por la cota de Ng y Hirschberg.

Ejercicios

1. Traza Boyer-Moore sobre C A B A A C A B A y escribe, después de cada paso, la partición en parejas y copias que promete el invariante.
2. **Misra-Gries.** Generaliza Boyer-Moore para encontrar todos los elementos que aparecen más de n/k veces, usando $k - 1$ pares candidato-contador. Enuncia el invariante, demuéstralo y di cuántas pasadas hacen falta.
3. Demuestra que con un alfabeto de dos símbolos el contador final de Boyer-Moore es la diferencia entre las dos cuentas, y deduce que el atajo de la sección 5 nunca se equivoca en ese caso.
4. **Compañeros de cuarto.** Exhibe cuatro personas con preferencias tales que ningún emparejamiento de a dos sea estable. ¿En qué lugar del algoritmo de la sección 7 se usa que hay dos lados, y por qué no se puede aplicar tal cual con un solo grupo?
5. Demuestra que Gale-Shapley le da a cada receptor su **peor** pareja válida.
6. Construye una instancia y un emparejamiento inicial en los que la reparación local de la sección 6 vuelva a un emparejamiento que ya visitó.
7. Escribe la búsqueda binaria con el invariante de la sección 1. Después cambia $lo = mid + 1$ por $lo = mid$, encuentra una entrada donde no termina y di en qué paso deja de bajar el potencial $hi - lo$.
8. Un receptor miente sobre sus preferencias. Construye una instancia de 3×3 en la que, con Gale-Shapley, consigue una pareja mejor que diciendo la verdad. (Un proponente, en cambio, nunca gana mintiendo. Es un teorema de Dubins y Freedman, de 1981, cuya demostración queda fuera del curso.)